

VentureMiner AI

Coding Standards

Document: 08-Engineering / 26_coding_standards

Version: v1.0

Date: 2026-07-20

Author: VentureMiner AI Documentation Team

Status: Approved

Project: VentureMiner AI — AI Venture Intelligence Platform

Table of Contents

Document 26 — Coding Standards

Table of Contents

1. Purpose & Scope
2. Languages
3. Project layout
4. Style (Python)
 - 4.1 Forbidden
5. Style (TypeScript)
 - 5.1 Forbidden
6. Naming
7. Comments & documentation
8. Error handling
9. Logging
10. Testing conventions
11. Code review
12. Dependencies
13. Performance & complexity
14. Security
15. AI-assisted code
16. Appendix
 - 16.1 Revision history
 - 16.2 Cross-references

Document 26 — Coding Standards

How we write code. Languages, style, testing, code review, and the shared idioms that make a multi-team codebase feel like one codebase.

Table of Contents

1. Purpose & Scope
2. Languages
3. Project layout
4. Style (Python)
5. Style (TypeScript)
6. Naming
7. Comments & documentation
8. Error handling
9. Logging
10. Testing conventions
11. Code review
12. Dependencies
13. Performance & complexity
14. Security
15. AI-assisted code
16. Appendix

1. Purpose & Scope

This document is the contract for how code is written. It is intentionally short; the goal is consistency, not prescription. Where this conflicts with a language's community conventions, community wins — unless this document explicitly overrides.

2. Languages

- Python 3.12 — for backend services, AI plane, scripts.
- TypeScript 5.x — for web app, BFF, public API client, Node services.
- SQL — declared migrations (Atlas).
- HCL — Terraform.
- Markdown / Mermaid — docs.

3. Project layout

```
ventureminer/
```

```
■■■ apps/
```

```

■ ■■■ web/ # Next.js
■ ■■■ docs-site/ # public docs
■■■ services/
■ ■■■ auth-svc/
■ ■■■ opportunity-svc/
■ ■■■ agent-runtime/
■ ■■■ ...
■■■ libs/
■ ■■■ ts/
■ ■ ■■■ ui/ # design system
■ ■ ■■■ api-client/
■ ■ ■■■ types/
■ ■■■ py/
■ ■■■ vma_common/ # shared Python utilities
■ ■■■ vma_agents/ # agent SDK
■■■ packages/
■ ■■■ contracts/ # shared types (pydantic / zod)
■■■ infra/
■ ■■■ terraform/
■■■ docs/ # this suite
■■■ tools/ # dev tools, codegen
■■■ README.md

```

Each service is a workspace with its own package.json / pyproject.toml, README, Dockerfile, and runbook stub.

4. Style (Python)

- Formatter: Black + isort (line length 100).
- Type checker: mypy strict.
- Lint: Ruff (replaces flake8, pyflakes, pylint).
- Imports: absolute; sorted by isort.
- Functions: small (< 50 lines), single responsibility, no hidden side effects.
- Async: use async end-to-end in IO-bound code.
- Type hints: required for public functions.
- Pydantic v2 for I/O models.

4.1 Forbidden

- print in production code (use logger).
- Bare except:.
- Mutable default arguments.

5. Style (TypeScript)

- Formatter: Prettier.
- Lint: ESLint (typescript-eslint, react, jsx-a11y).
- Type checker: tsc strict.
- Imports: absolute via path alias; no relative traversals.
- React: function components only; hooks for state.
- State: React Query for server state; Zustand for UI state.
- Errors: typed error classes; no any in catch.

5.1 Forbidden

- any outside of explicit, justified type-narrowing.
- Inline styles (use the design system).
- Direct DOM access in React.
- useEffect for derived state.

6. Naming

- Files: snake_case.py, kebab-case.ts/tsx.
- Classes: PascalCase.
- Functions / variables: snake_case (Python), camelCase (TypeScript).
- Constants: UPPER_SNAKE_CASE.
- Booleans: is_X, has_X, should_X.
- Acronyms: 2-letter: uppercase (IOError); 3+ letter: PascalCase (HttpClient).

7. Comments & documentation

- Comments explain why, not what.
- Docstrings (Python) follow PEP 257; JSDoc for public TS APIs.
- Module README for every service.
- Public APIs have at least one example.
- No commented-out code — delete it; the VCS remembers.

8. Error handling

- Errors are values. Raise or return, don't ignore.
- Typed exceptions in Python; error classes in TS.
- Never swallow an exception without a reason and a comment.

9. Logging

- Logger name = module name (`__name__`).
- Levels: DEBUG (dev only), INFO (auditable events), WARNING (recoverable), ERROR (failed operation), CRITICAL (page).
- No PII in logs. Ever.
- Structured fields preferred over format strings.
- No f-strings in logger calls — pass args, not strings.

10. Testing conventions

See Document 22.

- Test file: `test_.py` (Python), `.test.ts` (TS).
- Test name: `test__`.
- Fixtures: minimal; explicit over implicit.
- Mocks: at the boundary; never the whole world.
- One assertion per test as a default; multi-assert only when coupled.

11. Code review

- 2+ reviewers for changes to auth, crypto, RLS, or AI plane.
- 1 reviewer for everything else.
- CODEOWNERS enforced.
- Reviewer checklist:
 - Correctness.
 - Tests cover the change.
 - No new lint or type errors.
 - No new security issues.
 - Documentation updated.
 - Backwards compatibility considered.
 - Observability considered (logs, metrics, traces).
- Bots don't count as reviewers for the 2-reviewer rule.

12. Dependencies

- Add a dependency only when necessary.
- Pin major + minor; allow patch updates via Dependabot.
- Review every Dependabot PR within 7 days; longer for major.

13. Performance & complexity

- Time complexity stated in a comment when not obvious.
- DB queries are reviewed for index usage and N+1 risk.
- Hot paths are benchmarked.
- Memory is bounded; no unbounded collections.
- Backpressure is explicit; we do not silently queue forever.

14. Security

- Secrets in code = P0 incident.
- No raw SQL; use the repository layer.
- No eval-style construction of code or SQL.
- No untrusted HTML rendered without sanitization.
- Crypto via standard library; never roll your own.
- Authz at the boundary; do not trust the client.

15. AI-assisted code

- AI assistance is welcome, but:
- All code is reviewed by a human.
- The author is responsible for the code, not the tool.
- Generated code is held to the same standards.
- Sensitive areas (auth, crypto, AI plane) require a second reviewer.
- Generated comments are reviewed for accuracy.

16. Appendix

16.1 Revision history

v0.5	2026-07-20	Doc Team	All sections drafted
v1.0	2026-07-20	Doc Team	First approved version

16.2 Cross-references

- TRD: Document 02.
- Testing: Document 22.
- Security: Document 21.

End of Document 26 — Coding Standards. Consistency is the cheapest performance optimization.